Cryptografie en hashes herkennen en begrijpen

Uitleg over zwaktes, aanvallen en veelgemaakte implementatiefouten

Opmerking

Dit document legt in begrijpelijke taal (niveau B2) uit hoe je verschillende soorten versleuteling en hashfuncties herkent, waarom ze zwak kunnen zijn, en welke fouten bij de **implementatie** vaak voor problemen zorgen. Het is bedoeld als leer- en naslagmateriaal, bijvoorbeeld voor een cursus of CTF-oefening op je eigen lab-omgeving. De inhoud is gebaseerd op een eerder stappenplan en gecontroleerd aan de hand van een uitgebreidere technische cheatsheet, en is aangevuld met extra zwaktes die vaak voorkomen in de praktijk.

Contents

1 Stappenplan: hoe herken je wat voor cryptografie je ziet? 2
- 1.1 Stap 1 – Kijk naar het formaat van de uitvoer . . . 2
- 1.2 Stap 2 – Kijk hoe de data verwerkt wordt . . . 2
- 1.3 Stap 3 – Onderzoek de operationele modus (bij block ciphers) . . . 2
- 1.4 Stap 4 – Controleer op hergebruik en structuurfouten . . . 2

2 Klassieke ciphers 3
- 2.1 Caesar-cipher . . . 3
- 2.2 Monoalfabetische substitutie . . . 3
- 2.3 Vigenère-cipher . . . 3

3 Stream ciphers 3
- 3.1 One-Time Pad (OTP) . . . 4
- 3.2 PRNG- of LCG-gebaseerde stream ciphers . . . 4
- 3.3 RC4 . . . 4
- 3.4 Salsa20 / ChaCha20 . . . 4
- 3.5 Algemene zwaktes van stream ciphers . . . 4

4 Block ciphers en operationele modi 5
- 4.1 DES (Data Encryption Standard) . . . 5
- 4.2 Triple DES (3DES) . . . 5
- 4.3 AES (Advanced Encryption Standard) . . . 5
- 4.4 Dubbele encryptie ($E_{k2}(E_{k1}(m))$) . . . 5
  - 4.4.1 ECB (Electronic Codebook) . . . 6
  - 4.4.2 CBC (Cipher Block Chaining) . . . 6
  - 4.4.3 CFB en OFB . . . 6
  - 4.4.4 CTR (Counter Mode) . . . 6
  - 4.4.5 XTS-AES . . . 6

5 Hashfuncties 6
- 5.1 Algemene aanvallen op hashfuncties . . . 6
- 5.2 Custom of zwakke zelfgebouwde hashes . . . 7
- 5.3 MD5, SHA-0, SHA-1 . . . 7
- 5.4 SHA-2 (SHA-224/256/384/512) . . . 7
- 5.5 SHA-3 . . . 7
- 5.6 Wachtwoordhashes: unsalted vs. trage gesalte hashes . . . 7
- 5.7 Voorspelbare hash-kettingen . . . 7

6 MAC's, authenticated encryption en tokens 8
- 6.1 Eenvoudige MAC via $H(\text{sleutel} \,\|\, \text{bericht})$ . . . 8
- 6.2 HMAC (Hash-based Message Authentication Code) . . . 8
- 6.3 Authenticated Encryption (AE) . . . 8
- 6.4 JSON Web Tokens (JWT) . . . 8
- 6.5 HTTP/RTSP Digest Authentication . . . 9

7 Extra: veelgemaakte implementatiefouten **9**
7.1 Hergebruik van IV of nonce (algemeen) . . . 9
7.2 Onveilige random-generatoren . . . 9
7.3 Hardcoded of hergebruikte sleutels . . . 10
7.4 Textbook RSA (RSA zonder padding) . . . 10
7.5 RSA-padding-oracle (Bleichenbacher-stijl aanvallen) . . . 10
7.6 Gedeelde priemfactoren bij RSA . . . 10
7.7 Timing-aanvallen bij vergelijkingen . . . 10
7.8 Replay-aanvallen . . . 11
7.9 Downgrade-aanvallen . . . 11

8 Snelle herkenningstabel **11**

9 Afsluiting **11**

1 Stappenplan: hoe herken je wat voor cryptografie je ziet?

Voordat je iets kunt aanvallen of beoordelen, moet je eerst weten *wat* je eigenlijk voor je hebt. Volg daarvoor deze vier stappen in volgorde.

1.1 Stap 1 – Kijk naar het formaat van de uitvoer

De lengte en vorm van een string vertellen vaak al heel veel.

- **Vaste-lengte hexadecimale string** (alleen tekens `0-9` en `a-f`): dit is bijna altijd een hash. De lengte verraadt het algoritme:

Lengte (hex-tekens)	Algoritme
32 (16 bytes / 128 bit)	MD5
40 (20 bytes / 160 bit)	SHA-1
56 (28 bytes / 224 bit)	SHA-224
64 (32 bytes / 256 bit)	SHA-256 of SHA3-256
96 (48 bytes / 384 bit)	SHA-384
128 (64 bytes / 512 bit)	SHA-512

- **Begint met een herkenbare prefix:** dit wijst op een wachtwoord-hashfunctie met ingebouwde salt: `$2a$`, `$2b$`, `$2y$` voor bcrypt; `$argon2` voor Argon2; `$5$` voor sha256crypt; `$6$` voor sha512crypt.
- **Drie delen gescheiden door punten** (`header.payload.signature`): dit is een JSON Web Token (JWT), gecodeerd in Base64URL.
- **Leesbare tekst met verschoven letters:** dit wijst op een klassieke cipher zoals Caesar, een monoalfabetische substitutie of Vigenère.

Voorbeeld

Je ziet de string `5f4dcc3b5aa765d61d8327deb882cf99`. Dat zijn 32 hex-tekens, dus 16 bytes: dit is zeer waarschijnlijk een MD5-hash (in dit geval toevallig de hash van het woord “password”).

1.2 Stap 2 – Kijk hoe de data verwerkt wordt

- **Stream cipher:** de ciphertext heeft *exact dezelfde lengte* als de plaintext. Er gebeurt een bitwise XOR tussen elke byte van het bericht en een byte van een sleutelstroom (keystream).
- **Block cipher:** de data wordt in vaste blokken verwerkt, meestal 8 bytes (64 bit, bijvoorbeeld DES) of 16 bytes (128 bit, bijvoorbeeld AES). De totale lengte van de ciphertext is daardoor een veelvoud van de blokgrootte, vaak met padding aangevuld.

1.3 Stap 3 – Onderzoek de operationele modus (bij block ciphers)

- **ECB:** zoek naar herhalende blokken in de ciphertext. Identieke plaintextblokken geven identieke ciphertextblokken.
- **CBC:** gebruikt een initialisatievector (IV). Het aanpassen van een byte in blok $i-1$ verandert de plaintext van blok i op een voorspelbare manier.
- **CTR:** gedraagt zich als een stream cipher doordat een teller steeds wordt versleuteld. Kwetsbaar bij hergebruik van de teller of nonce.

1.4 Stap 4 – Controleer op hergebruik en structuurfouten

- **Hergebruik van sleutel of nonce:** als twee ciphertexts met dezelfde sleutel/nonce zijn gemaakt, geldt $c_1 \oplus c_2 = p_1 \oplus p_2$. Als je één plaintext kent, kun je de andere direct berekenen.
- **Zelfgeschreven (custom) functies:** bevat de code lussen met omkeerbare bewerkingen zoals XOR, rotaties (`rol`/`ror`), optellingen of bitverschuivingen? Dan gaat het waarschijnlijk om een zelfgemaakte, zwakke cipher of hash die je stap voor stap kunt terugrekenen.

Voorbeeld

Je onderschept twee berichten die met dezelfde stream-cipher-sleutel zijn versleuteld. Je kent de inhoud van het eerste bericht (bijvoorbeeld een standaard-begroeting). Door $c_1 \oplus p_1$ te berekenen, vind je de keystream. Die keystream kun je daarna XOR'en met c_2 om het tweede, onbekende bericht te lezen. Dit is precies waarom nonce-hergebruik zo gevaarlijk is.

2 Klassieke ciphers

2.1 Caesar-cipher

Hoe herken je dit?

Eenvoudige tekst waarbij elke letter met een vaste waarde is verschoven (bijvoorbeeld A wordt D, B wordt E, enzovoort).

Waarom is dit zwak?

Er zijn maar 25 mogelijke sleutels. Dat is zo weinig dat je ze allemaal binnen een fractie van een seconde kunt uitproberen (brute-force), of je gebruikt frequentie-analyse: de letter die het vaakst voorkomt in de ciphertext is waarschijnlijk de versleutelde vorm van de meest voorkomende letter in de brontaal (in het Nederlands en Engels meestal de "e").

2.2 Monoalfabetische substitutie

Hoe herken je dit?

Elke letter van het alfabet is vervangen door een andere, vaste letter, maar niet via een simpele verschuiving zoals bij Caesar.

Waarom is dit zwak?

De sleutelruimte is nu wel groot (26! mogelijke combinaties), maar de *relatieve frequentie* van letters verandert niet: een letter die vaak voorkomt in de brontaal, komt ook vaak voor in de ciphertext – alleen onder een andere naam. Met statistische frequentie-analyse (letterfrequenties, veelvoorkomende letterparen) is de mapping meestal binnen korte tijd te achterhalen.

2.3 Vigenère-cipher

Hoe herken je dit?

Een herhalend trefwoord wordt gebruikt om meerdere Caesar-verschuivingen na elkaar toe te passen op de tekst.

Waarom is dit zwak?

Omdat de sleutel herhaald wordt, ontstaan er periodieke patronen in de ciphertext. Met de Kasiski-test (zoeken naar herhalende reeksen tekens) of de *index of coincidence* kun je de sleutellengte achterhalen. Zodra je de lengte kent, splits je de tekst op in evenveel kolommen als de sleutel lang is; elke kolom is dan gewoon een Caesar-cipher die je apart met frequentie-analyse kunt kraken.

Voorbeeld

Stel, de sleutel is 4 tekens lang. Dan hoort letter 1, 5, 9, 13, ... van de ciphertext allemaal bij dezelfde verschuiving (kolom 1). Je behandelt die letters dus als één aparte Caesar-cipher, los van kolom 2, 3 en 4.

3 Stream ciphers

3.1 One-Time Pad (OTP)

Hoe herken je dit?
Een volledig willekeurige sleutelstroom, precies zo lang als het bericht, gecombineerd via bitwise XOR.

Waarom is dit zwak?
Wiskundig gezien is OTP de enige vorm van versleuteling die perfecte geheimhouding biedt – *mits* drie strikte voorwaarden kloppen: de sleutel is echt willekeurig, minstens even lang als het bericht, en wordt **nooit** hergebruikt. In de praktijk is dit bijna onhaalbaar om vol te houden, en zodra de sleutel ook maar één keer hergebruikt wordt, stort de beveiliging volledig in (zie de algemene stream-cipherzwakte hieronder).

3.2 PRNG- of LCG-gebaseerde stream ciphers

Hoe herken je dit?
Een *Linear Congruential Generator* (LCG) of vergelijkbare pseudowillekeurige generator zet een korte seed om in een lange keystream, meestal via een formule als `state = (A * state + C) mod M`.

Waarom is dit zwak?
Een LCG is wiskundig voorspelbaar: zodra een aanvaller een paar opeenvolgende outputbytes kent (en de constanten A, C, M bekend of simpel te raden zijn), kan de interne staat worden teruggerekend. Is de seed-ruimte klein, dan is zelfs gewoon brute-force vaak al snel genoeg.

3.3 RC4

Hoe herken je dit?
Historische stream cipher zonder apart nonce-mechanisme, vroeger veel gebruikt in bijvoorbeeld WEP en oudere TLS-verbindingen.

Waarom is dit zwak?
RC4 heeft een structurele, statistische vertekening (bias) in de eerste bytes van de keystream. Door heel veel sessies te verzamelen, kunnen aanvallers (zoals bij de bekende NOMORE-aanval) hierdoor bijvoorbeeld cookies of andere geheime data achterhalen. RC4 wordt om deze reden al jaren afgeraden.

3.4 Salsa20 / ChaCha20

Hoe herken je dit?
Moderne, snelle stream ciphers die een geheime sleutel combineren met een publiek nonce.

Waarom is dit zwak?
Het algoritme zelf is sterk, maar **hergebruik van (sleutel, nonce)** is fataal: net als bij elke stream cipher ontstaat dan de klassieke XOR-zwakte (zie hieronder).

3.5 Algemene zwaktes van stream ciphers

Waarom is dit zwak?
Keystream- of nonce-hergebruik: als twee berichten met dezelfde keystream (dus dezelfde sleutel én nonce) zijn versleuteld, geldt $c_1 \oplus c_2 = p_1 \oplus p_2$. Is één plaintext bekend, dan is de keystream – en dus ook het tweede bericht – volledig te achterhalen.
Vormbaarheid (malleability) / bit-flipping attack: omdat $c = p \oplus k$ geldt, kan een aanvaller de ciphertext XOR'en met een zelfgekozen masker ($c' = c \oplus t$) om de plaintext op een *exact gekozen* manier te wijzigen, zonder de sleutel te kennen. Dit wordt voorkomen door altijd een MAC (integriteitscontrole) toe te voegen aan de versleuteling.

Voorbeeld

Een factuur bevat het bedrag "1000" op een vaste positie in het bericht. Een aanvaller die de positie kent, maar niet de sleutel, kan met een bit-flip het bedrag veranderen in "9000" zonder ooit de inhoud te hoeven "ontcijferen". Dit laat zien dat versleuteling zonder integriteitscontrole (MAC) de data wel geheim houdt, maar niet beschermt tegen manipulatie.

4 Block ciphers en operationele modi

4.1 DES (Data Encryption Standard)

Hoe herken je dit?

Verwerkt data in blokken van 64 bit, met een sleutel van 56 bit.

Waarom is dit zwak?

56 bit is tegenwoordig veel te weinig: gespecialiseerde hardware (of zelfs een cluster van gewone computers) kan de volledige sleutelruimte binnen enkele uren doorzoeken. DES wordt daarom als onveilig beschouwd.

4.2 Triple DES (3DES)

Hoe herken je dit?

Past het DES-algoritme drie keer na elkaar toe, meestal met twee of drie verschillende sleutels.

Waarom is dit zwak?

3DES is trager dan AES, en de effectieve sleutelsterkte ligt door specifieke cryptanalytische aanvallen (zoals meet-in-the-middle, zie verderop) lager dan je op basis van de totale sleutellengte zou verwachten.

4.3 AES (Advanced Encryption Standard)

Hoe herken je dit?

Ondersteunt sleutels van 128, 192 of 256 bit, met een vaste blokgrootte van 128 bit (16 bytes).

Waarom is dit zwak?

AES zelf is, bij correct gebruik, wiskundig sterk en niet praktisch te breken. Problemen ontstaan vrijwel altijd door een **verkeerd gekozen modus** (zoals ECB) of door slecht **sleutelbeheer** (bijvoorbeeld een hardcoded of hergebruikte sleutel) – niet door het algoritme zelf.

4.4 Dubbele encryptie ($E_{k2}(E_{k1}(m))$)

Hoe herken je dit?

Data wordt twee keer na elkaar versleuteld met twee (vaak kortere) sleutels, in de veronderstelling dat dit de beveiliging kwadratisch versterkt.

Waarom is dit zwak?

Dit is kwetsbaar voor de **meet-in-the-middle-aanval**. In plaats van 2^{2n} pogingen (wat je zou verwachten bij twee losse n-bit sleutels) kost het kraken slechts ongeveer 2×2^n geheugen- en rekenstappen: je versleutelt een bekend plaintextblok met alle mogelijke eerste sleutels en slaat de resultaten op, en ontsleutelt het bijbehorende ciphertextblok met alle mogelijke tweede sleutels totdat je een match vindt in de tabel.

4.4.1 ECB (Electronic Codebook)

Waarom is dit zwak?
Elk blok wordt onafhankelijk en met dezelfde sleutel versleuteld. Identieke plaintextblokken leveren dus altijd identieke ciphertextblokken op, waardoor patronen en structuren in de oorspronkelijke data (denk aan een afbeelding met grote egale vlakken) zichtbaar blijven in de ciphertext. Dit staat bekend als het "ECB-penguin"-effect.

4.4.2 CBC (Cipher Block Chaining)

Waarom is dit zwak?
Elk plaintextblok wordt vóór versleuteling ge-XOR'd met het *vorige* ciphertextblok. Dit heeft twee gevolgen:

- Een aanvaller die bits in een ciphertextblok aanpast, kan daarmee op een voorspelbare manier het *volgende* plaintextblok wijzigen (bit-flipping), weliswaar ten koste van het vervormen van het blok zelf.
- Als een server bij het ontsleutelen een ander foutbericht geeft voor "verkeerde padding" dan voor andere fouten, ontstaat een **padding oracle**: door gericht bytes in het voorlaatste blok te wijzigen en de serverreacties te observeren, kan de hele plaintext blok voor blok worden ontcijferd, zonder de sleutel te kennen.

4.4.3 CFB en OFB

Waarom is dit zwak?
Deze modi zetten een block cipher om in een stream cipher. OFB is net als andere stream ciphers gevoelig voor bit-manipulatie; CFB heeft bovendien last van foutenpropagatie: een fout in één ciphertextblok beïnvloedt ook de ontcijfering van het volgende blok.

4.4.4 CTR (Counter Mode)

Waarom is dit zwak?
Een oplopende tellerwaarde wordt versleuteld en als keystream gebruikt – functioneel dus een stream cipher. Wordt een teller (of nonce) hergebruikt voor een nieuw bericht met dezelfde sleutel, dan ontstaat precies dezelfde XOR-zwakte als bij een gewone stream cipher: alle vertrouwelijkheid vervalt.

4.4.5 XTS-AES

Hoe herken je dit?
Speciaal ontworpen voor schijfversleuteling (disk encryption), met een extra "tweak"-waarde gebaseerd op de sectorpositie.

Waarom is dit zwak?
De tweak voorkomt dat identieke sectoren dezelfde ciphertext opleveren en maakt het lastiger om sectoren te verplaatsen zonder detectie, maar XTS biedt **geen** integriteitsbescherming (geen MAC): gerichte bit-manipulatie binnen een sector blijft mogelijk.

5 Hashfuncties

5.1 Algemene aanvallen op hashfuncties

Waarom is dit zwak?
Preimage attack: gegeven een hash h, een invoer y vinden zodat $H(y) = h$. Kost gemiddeld 2^{m-1} pogingen bij een m-bit hash.
Second preimage attack: gegeven een invoer x, een *andere* invoer $y \neq x$ vinden met $H(y) = H(x)$. Net zo duur als een gewone preimage-aanval.
Collision attack (verjaardagparadox / birthday attack): twee *willekeurige* invoeren vinden met

dezelfde hash. Dankzij de verjaardagparadox kost dit slechts ongeveer $2^{m/2}$ pogingen in plaats van 2^m – een enorme versnelling.

Voorbeeld

Bij een 32-bit hash is een gewone preimage-aanval 2^{31} pogingen, maar een botsing vinden kost maar ongeveer $2^{16} = 65\,536$ pogingen. Dat laatste is binnen enkele seconden haalbaar op een gewone laptop, wat laat zien hoeveel sterker de verjaardagparadox werkt dan een preimage-zoektocht.

5.2 Custom of zwakke zelfgebouwde hashes

Hoe herken je dit?

Zelfgeschreven hashfuncties in code, met lussen die XOR, bit-rotaties of optellingen gebruiken.

Waarom is dit zwak?

Als *alle* stappen omkeerbaar zijn (zoals XOR, rotate, optellen modulo 2^n), kan de hash volledig worden teruggerekend door de operaties in omgekeerde volgorde en met hun inverse uit te voeren (laatste stap eerst). Zitten er niet-omkeerbare operaties in zoals `AND` of `OR`, dan kan vaak nog steeds een oplossing gevonden worden met een SAT/SMT-solver zoals **z3**.

5.3 MD5, SHA-0, SHA-1

Waarom is dit zwak?

Deze verouderde algoritmes zijn wiskundig gebroken en kwetsbaar voor collision-aanvallen die praktisch uitvoerbaar zijn. SHA-1 wordt al sinds 2005 afgeraden voor nieuwe toepassingen.

5.4 SHA-2 (SHA-224/256/384/512)

Waarom is dit zwak?

SHA-1 en SHA-2 zijn gebouwd op de Merkle-Damgård-constructie. Dat maakt ze kwetsbaar voor **length extension attacks** zodra ze direct, zonder HMAC, als MAC worden gebruikt (zie het hoofdstuk over MAC's hieronder).

5.5 SHA-3

Waarom is dit zwak?

SHA-3 gebruikt een sponsconstructie in plaats van Merkle-Damgård, en is daardoor **niet** kwetsbaar voor length extension attacks.

5.6 Wachtwoordhashes: unsalted vs. trage gesalte hashes

Waarom is dit zwak?

Onsalted hashes (bijvoorbeeld kale SHA-1 of SHA-256 zonder salt) geven voor hetzelfde wachtwoord bij elke gebruiker exact dezelfde hash. Dat maakt ze extreem kwetsbaar voor **rainbow tables** (vooraf berekende hash-tabellen) en **dictionary attacks** (woordenboekaanvallen). Daarnaast zijn standaard-hashfuncties zoals SHA-256 juist ontworpen om snel te zijn, wat ze ongeschikt maakt voor wachtwoorden: een aanvaller kan met GPU's miljarden combinaties per seconde proberen.
Oplossing: gebruik altijd een unieke, willekeurige salt per wachtwoord (voorkomt hergebruik van rainbow tables) én een *trage*, geheugenintensieve hashfunctie zoals **Bcrypt**, **Scrypt** of **Argon2**, zodat brute-force vertraagd wordt.

5.7 Voorspelbare hash-kettingen

Hoe herken je dit?

Een systeem genereert eenmalige codes via `volgende_code = H(huidige_code)`.

Waarom is dit zwak?
Iedereen die de huidige code kent, kan direct alle toekomstige codes berekenen door de hashfunctie herhaaldelijk toe te passen – er zit geen extra geheim of onvoorspelbaar element in de keten.

6 MAC's, authenticated encryption en tokens

6.1 Eenvoudige MAC via $H(\text{sleutel} \,\|\, \text{bericht})$

Waarom is dit zwak?
Dit patroon is kwetsbaar voor de **length extension attack** bij Merkle-Damgård-hashes (MD5, SHA-1, SHA-2). Een aanvaller die de digest en de *lengte* van de geheime sleutel kent (de sleutel zelf hoeft hij niet te kennen), kan extra data achter het oorspronkelijke bericht plakken en daar een geldige nieuwe hash voor berekenen.

Voorbeeld
Een server berekent `sha256(secret + "actie=lezen")` als handtekening. Een aanvaller die deze hash en de berichtlengte kent, kan – zonder de sleutel te kennen – een geldige handtekening berekenen voor `"actie=lezen" + padding + "&rol=admin"`. De server accepteert dit bericht dan alsof het door de legitieme afzender is ondertekend.

6.2 HMAC (Hash-based Message Authentication Code)

Waarom is dit zwak?
HMAC is specifiek ontworpen om dit probleem op te lossen door de hash twee keer toe te passen op een slimme manier. HMAC is **niet** kwetsbaar voor length extension attacks. Aanvallen op HMAC vereisen een online brute-force van de sleutel (2^n pogingen) of het verzamelen van een onhaalbaar grote hoeveelheid live netwerkverkeer voor een botsing ($2^{n/2}$ blokken). HMAC biedt wel geen *non-repudiation* (onweerlegbaarheid), omdat beide partijen dezelfde geheime sleutel delen en dus beiden berichten konden hebben ondertekend.

6.3 Authenticated Encryption (AE)

Hoe herken je dit?
Combinatie van versleuteling (vertrouwelijkheid) en een MAC (integriteit/authenticiteit) in één schema. Bekende volgorde-varianten: `H -> E` (bijvoorbeeld WEP, inmiddels bewezen onveilig), `A -> E` (SSL/TLS), `E -> A` (IPSec) en `E + A` (SSH), naast de moderne NIST-standaarden **CCM** en **GCM**.

6.4 JSON Web Tokens (JWT)

Hoe herken je dit?
Een string van drie met punten gescheiden Base64URL-delen: `header.payload.signature`.

Waarom is dit zwak?
- `alg=none-attack`: als de server het `alg`-veld blind vertrouwt, kan een aanvaller dit op `"none"` zetten; de server slaat dan de handtekeningcontrole over.
- **Zwak HMAC-secret:** bij HS256 met een eenvoudig wachtwoord kan het geheim offline met een woordenboek worden gekraakt.
- **Algorithm confusion (RS256 → HS256):** als de server beide algoritmes accepteert, kan een aanvaller een HS256-token maken en de *publieke* RSA-sleutel van de server (die iedereen kent) gebruiken als HMAC-geheim. De server gebruikt die publieke sleutel dan per ongeluk om de vervalste handtekening te "verifiëren", en accepteert het token.
- `kid`-**injectie:** de `kid` (key ID) header wordt misbruikt voor path traversal of SQL-injectie om te beïnvloeden welke sleutel de server gebruikt.
- `jku`/`x5u` **SSRF:** de server haalt de publieke sleutel blindelings op van een URL die de aanvaller zelf heeft opgegeven.

6.5 HTTP/RTSP Digest Authentication

Hoe herken je dit?
Maakt gebruik van MD5-hashes van gebruikersnaam, realm, wachtwoord, URI en nonce.

Waarom is dit zwak?
Is het wachtwoord van de gebruiker zwak, dan kan een aanvaller het offline bruteforcen door met een bekende realm, nonce, methode en URI de verwachte `response`-hash te vergelijken met wat er over het netwerk is gestuurd.

7 Extra: veelgemaakte implementatiefouten

Opmerking
Dit hoofdstuk stond niet in de oorspronkelijke tekst. Het vult aan met fouten die in de praktijk minstens zo vaak voorkomen als de "klassieke" zwaktes hierboven – vaak zit het probleem namelijk niet in het algoritme zelf, maar in hoe het wordt toegepast.

7.1 Hergebruik van IV of nonce (algemeen)

Extra (niet in de oorspronkelijke tekst)
Niet alleen bij stream ciphers of CTR-mode is hergebruik gevaarlijk. Ook bij CBC moet de IV uniek en onvoorspelbaar zijn per bericht: een voorspelbare of hergebruikte IV maakt bepaalde aanvallen (zoals het detecteren of twee berichten met hetzelfde eerste blok beginnen) mogelijk.

Waarom is dit zwak?
AES-GCM nonce-hergebruik is extra catastrofaal: GCM combineert versleuteling en authenticatie. Wordt dezelfde (sleutel, nonce)-combinatie twee keer gebruikt, dan kan een aanvaller niet alleen de keystream berekenen (zoals bij elke stream cipher), maar in veel gevallen ook de interne authenticatiesleutel van GCM afleiden. Daarmee kan hij vervolgens *elk willekeurig bericht* vervalsen en laten accepteren als authentiek, niet alleen het ene bericht waarvoor de nonce werd hergebruikt.

Voorbeeld
Een applicatie gebruikt AES-GCM maar initialiseert de nonce steeds opnieuw als `0000...0` in plaats van een echte, willekeurige of oplopende waarde. Elk bericht gebruikt daardoor dezelfde nonce, wat de volledige authenticiteitsgarantie van GCM teniet doet.

7.2 Onveilige random-generatoren

Extra (niet in de oorspronkelijke tekst)
Veel programmeertalen hebben twee soorten randomgeneratoren: een snelle, voorspelbare (bijvoorbeeld Python's `random`-module, gebaseerd op een Mersenne Twister) en een cryptografisch veilige (bijvoorbeeld Python's `secrets`-module of `os.urandom`).

Waarom is dit zwak?
Wordt een *niet*-cryptografische generator gebruikt voor sleutels, tokens, wachtwoord-resetlinks of session-ID's, dan kan een aanvaller die enkele outputs ziet vaak de interne staat van de generator reconstrueren en daarmee toekomstige (of soms zelfs eerdere) waarden voorspellen – net als bij de LCG-zwakte hierboven, maar dan in een echt productiesysteem.

7.3 Hardcoded of hergebruikte sleutels

Extra (niet in de oorspronkelijke tekst)

Een sleutel die letterlijk in de broncode staat (en dus terug te vinden is via decompilatie, een publieke Git-repository, of simpelweg `strings` op een binary), of een sleutel die voor meerdere, losse doeleinden wordt hergebruikt (bijvoorbeeld dezelfde sleutel voor encryptie én voor het signeren van tokens).

Waarom is dit zwak?

Lekt de sleutel op één plek (bijvoorbeeld via een gelekte broncode-repo), dan zijn automatisch *alle* toepassingen die dezelfde sleutel gebruiken gecompromitteerd. Daarnaast kan het combineren van dezelfde sleutel voor verschillende algoritmes soms leiden tot cross-protocol-aanvallen, waarbij een handtekening of ciphertext uit het ene protocol hergebruikt kan worden om het andere protocol te misleiden.

7.4 Textbook RSA (RSA zonder padding)

Extra (niet in de oorspronkelijke tekst)

RSA wordt rechtstreeks toegepast op de data ($c = m^e \bmod N$), zonder een paddingschema zoals OAEP.

Waarom is dit zwak?

Naast de **kleine-exponent-aanval** (als $e = 3$ en het bericht klein genoeg is dat $m^e < N$, dan is c gewoon m^3 zonder modulaire reductie, en kun je de plaintext vinden door simpelweg de derdemachtswortel van c te trekken) is textbook RSA ook **deterministisch**: hetzelfde bericht geeft altijd dezelfde ciphertext. Een aanvaller die vermoedt wat het bericht is (bijvoorbeeld "JA" of "NEE"), kan dat zelf versleutelen en vergelijken met de onderschepte ciphertext.

7.5 RSA-padding-oracle (Bleichenbacher-stijl aanvallen)

Extra (niet in de oorspronkelijke tekst)

RSA met PKCS#1 v1.5-padding, waarbij de server een ander antwoord of een andere foutmelding geeft afhankelijk van of de padding na ontsleuteling geldig is.

Waarom is dit zwak?

Dit is het RSA-equivalent van de CBC padding-oracle-aanval: door duizenden aangepaste ciphertexts te sturen en te kijken welke een "geldige padding"-reactie krijgen, kan een aanvaller geleidelijk het hele bericht (of zelfs de sleutel waarmee een sessie wordt opgezet) reconstrueren, zonder de private sleutel zelf te kennen.

7.6 Gedeelde priemfactoren bij RSA

Waarom is dit zwak?

Als twee verschillende RSA-sleutels ($N_1 = p \cdot q_1$ en $N_2 = p \cdot q_2$) per ongeluk dezelfde priemfactor p delen – vaak het gevolg van een zwakke randomgenerator bij het genereren van sleutels – dan kan een aanvaller die factor direct berekenen via $\gcd(N_1, N_2) = p$. Daarmee zijn beide sleutels in één keer gebroken.

7.7 Timing-aanvallen bij vergelijkingen

Extra (niet in de oorspronkelijke tekst)

Een applicatie vergelijkt twee hashes, tokens of wachtwoorden met een standaard gelijkheidsoperator die stopt zodra de eerste afwijkende byte wordt gevonden.

Waarom is dit zwak?

Het kleine tijdsverschil tussen "meteen stoppen bij byte 1" en "pas stoppen bij byte 20" is in theorie (en soms ook in de praktijk, vooral lokaal of met voldoende metingen) meetbaar. Een aanvaller kan daardoor byte voor byte de juiste waarde reconstrueren. De oplossing is een *timing-neutrale* vergelijkingsfunctie, zoals `hmac.compare_digest` in Python, die altijd evenveel tijd kost ongeacht waar het verschil zit.

7.8 Replay-aanvallen

Extra (niet in de oorspronkelijke tekst)

Een geldig, eerder verstuurd bericht of token wordt gewoon opnieuw verzonden.

Waarom is dit zwak?

Als een systeem geen bescherming heeft tegen hergebruik – zoals een tijdstempel met een kort geldigheidsvenster, een oplopend volgnummer, of een eenmalig te gebruiken nonce die wordt bijgehouden – dan kan een aanvaller die een bericht onderschept het simpelweg opnieuw afspelen en dezelfde actie nogmaals laten uitvoeren (bijvoorbeeld een betaling of een login). Encryptie en een geldige handtekening alleen beschermen hier niet tegen: het bericht is immers echt geldig, het wordt alleen te vaak gebruikt.

7.9 Downgrade-aanvallen

Extra (niet in de oorspronkelijke tekst)

Een systeem ondersteunt zowel sterke als zwakke algoritmes of protocolversies, en laat de keuze (gedeeltelijk) over aan de client of aan onversleutelde onderhandeling vooraf.

Waarom is dit zwak?

Een aanvaller die het verkeer kan beïnvloeden (man-in-the-middle), kan de onderhandeling manipuleren zodat beide partijen denken een sterk algoritme te gebruiken terwijl in werkelijkheid een zwakke, makkelijker te breken variant wordt ingezet. Dit is dezelfde onderliggende logica als bij JWT `algorithm confusion` hierboven, maar dan op protocolniveau.

8 Snelle herkenningstabel

Wat je ziet	Waarschijnlijk type / vervolgstap
32 hex-tekens	MD5
40 hex-tekens	SHA-1
64 hex-tekens	SHA-256 of SHA3-256
128 hex-tekens	SHA-512
`$2a$`/`$2b$`/`$2y$`	bcrypt
`$argon2`	Argon2
`header.payload.signature`	JWT
Ciphertext even lang als plaintext	stream cipher
Ciphertext = veelvoud van 8 of 16 bytes	block cipher (DES resp. AES)
Herhalende 16-byte-blokken in ciphertext	ECB-modus, patroonlek
Fout verschilt bij ongeldige padding	CBC padding oracle
Zelfde key/nonce op 2 berichten	XOR-keystream-aanval
Zelfgeschreven hash met XOR/rotate	inverse-functie bouwen of z3
Server geeft andere timing bij vergelijken	timing-aanval
GCM-nonce lijkt constant of voorspelbaar	nonce-hergebruik, kritiek lek

9 Afsluiting

Bij vrijwel elke cryptografische zwakte komt het neer op een van deze drie oorzaken: de sleutelruimte is te klein (brute-force), een sleutel of nonce is hergebruikt (XOR-aanval), of de gekozen modus/implementatie klopt niet (ECB/CBC/padding/length-extension). Loop bij een nieuw geval dus altijd eerst het stappenplan uit hoofdstuk 1 af voor je verder zoekt.

Opmerking

Gebruik deze kennis alleen op je eigen lab-omgeving of op systemen waarvoor je uitdrukkelijk toestemming hebt.